

From pixels to phone

PARSeq explained visually, then turned into an engineering workflow.

PARSeq

READ THE IMAGE.

UNDERSTAND THE SYSTEM.

H

O

T

E

L

Visual evidence + permitted text context

HOTEL -> HOTEL is possible, never guaranteed.

Built for your next team task

You are joining a Document AI team. The useful skill is tracing a bad result from camera pixels to model tensors to the phone runtime, then changing one thing and measuring the effect. This handbook builds that skill from zero.

What these 22 pages give you

A mental model, real masks, a code reading map, model and decoding variants, and a practical path through export, accuracy, latency, memory and failure analysis. Simple English carries the detail; short Hinglish hooks help it stick.

A handbook + a lab, not a certification

Production competence comes from running the experiments. Use this as your launchpad: read, predict the outcome, inspect the official code, then reproduce the result. No device performance or proprietary Samsung pipeline is assumed.

Sources: [P](#) [R](#)

Choose your route

Tap a chapter tile, use the PDF bookmarks, or read from page 3.

01 / BUILD INTUITION

Pages 3-7

OCR, tensors, patches, attention, encoder

02 / OPEN THE MODEL

Pages 8-13

Decoder, masks, training, decoding, variants

03 / MAKE IT RELIABLE

Pages 14-16

Code baseline, datasets, multilingual, metrics

04 / TAKE IT ON DEVICE

Pages 17-20

Export, runtime, quantization, profiling

05 / PRACTISE & CHECK

Pages 21-22

Answers, failure cases, lab and sources

How to use the interaction

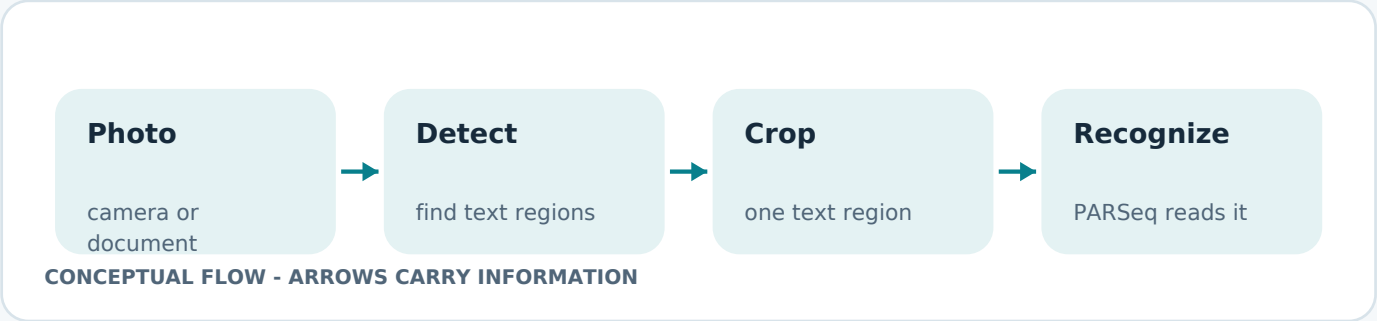
Gold **TRY** strips ask you to predict an outcome. Tap **Answer** to jump to page 21; each answer links back. Page 22 has a fillable lab checklist. Internal links work in many PDF viewers; if a preview ignores links or forms, open the downloaded PDF in a full reader.

Three labels you should keep separate

Verified implementation: what the official configuration/code does. **Conceptual illustration:** a simplified diagram or invented example. **Engineering proposal:** a mobile experiment you must validate. Runtime guidance was checked on 7 October 2026.

Where PARSeq fits

OCR is a system. PARSeq is its cropped-text recognizer.



Detection and recognition answer different questions

A detector asks **where is the text?** and returns boxes or polygons. A recognizer asks **what does this crop say?** and returns a character sequence. A page with 40 text regions may require 40 recognizer inputs. PARSeq alone does not identify tables, establish reading order or extract invoice fields.

The crop is already an important decision

Clipping or heavy background can break recognition. **Deskewing** rotates text toward its reading direction; **rectification** warps its shape toward a readable rectangle. Curved text may need a different method. Test these upstream choices with the recognizer.

Recognizer family	Main difference	Candidate experiment
CRNN / CTC	CNN learns local visual filters; RNN mixes sequence context. CTC merges repeats then removes blanks.	Test as a compact baseline on exact strings.
ABINet	Vision predictions + a separate bidirectional language refinement model.	Test whether extra compute improves your data.
TrOCR	Pretrained image/text Transformers; wordpiece (subword token) generation.	Test document/handwriting checkpoints and mobile cost.
PARSeq	Character position queries + permutation-trained decoder.	Compare NAR/AR/refinement with shared weights.

Hinglish hook

Detector jagah batata hai; PARSeq us jagah ka text padhta hai. Comparison experiments above are engineering proposals, not universal model rankings. Debug the exact model crop.

Numbers before networks

Pixel -> tensor -> embedding -> parameter: four different things.



A tensor is a shaped array of numbers

B is batch size: how many crops are processed together. **3** is RGB channels. **32,128** are image height and width in the common setup. For $B=1$ the input contains $3 \times 32 \times 128 = \mathbf{12,288}$ values. Layout matters: NCHW and NHWC store the same conceptual image with different dimension orders.

An embedding is a learned numerical representation

A patch or character is mapped to a vector: a list of D numbers. A vector is not a human-readable label, and its coordinates do not each mean 'curve' or 'edge'. **Parameters** are the persistent learned numbers used to make that mapping. **Activations** are temporary intermediate values produced for this particular input.

Token	Meaning	Where it belongs
BOS	Start / null text context	Input context; not a predicted character
EOS	End of the word	Classifier output; stops decoding
PAD	Unused sequence slot	Batch padding; ignored in training loss

Charset is an interface contract

With 94 ordinary characters, the output head has **95 classes** including EOS. Input embeddings also need BOS and PAD: **97 token IDs** in that setup. Do not confuse output class count with embedding vocabulary size. A 25-character limit leaves 26 output slots including EOS.

TRY 01 Is [1,26,95] one word, 26 words, or 95 words?

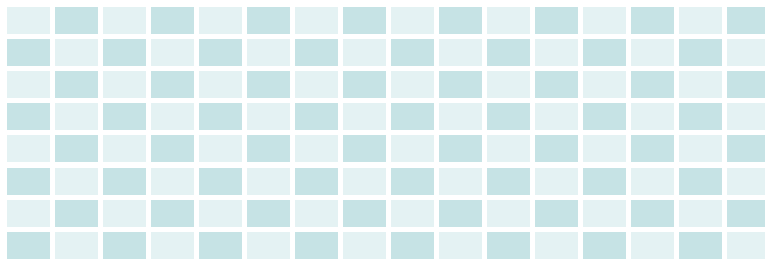
[Answer >](#)

Sources: [C1](#) [C4](#) [C7](#)

How an image becomes tokens

A patch is a spatial piece. A token is its numerical representation.

32 x 128 image, divided into 4 x 8 patches



8 rows x 16 columns = 128 patches

128 tokens

each has D=384 values

Patch embedding: the actual arithmetic

Each RGB patch contains $4 \times 8 \times 3 = 96$ values. Flattening makes a 96-element vector. A learned linear projection maps it to 384 values: $\mathbf{e} = \text{patch_vector} \times \mathbf{W} + \mathbf{b}$, with $\mathbf{W}$ shaped [96,384]. The image produces [B,128,384]. A convolution with kernel and stride equal to patch size can implement this projection.

Position tells the encoder where a patch came from

Attention alone does not supply the original grid order. A learned spatial position vector is added to each patch token. The encoder can then combine appearance with location. These **image position embeddings** differ from the decoder's **output character position queries**: one labels image patches, the other labels text slots.

Why resolution is a model choice

Making input width larger while keeping patches fixed creates more tokens. This can preserve detail in long text, but changes the positional embedding and runtime contract. With width 256, there would be 256 patches. You cannot assume a checkpoint trained at width 128 will accept or benefit from that change without adapting and validating it.

Inspect, do not invent alignment

Patch 17 is not guaranteed to equal character 2. Letters can span several patches, and a patch can contain background or portions of multiple letters. The decoder learns to combine useful visual tokens.

Attention without mystery

Find relevant sources, score them, then mix their information.

TOY EXAMPLE: one target asks three image patches

patch A		0.70
patch B		0.20
patch C		0.10

Weights sum to 1. These are illustrative, not measured.

Query, key and value are learned projections

Q (query): what this target is looking for. **K (key):** how each source can match that request. **V (value):** the information each source contributes. Learned matrices produce Q, K and V from token vectors. The names are a useful analogy; they are all arrays of numbers.

```
scores = (Q @ K.T) / sqrt(d_head) + mask
weights = softmax(scores, axis=-1)
output = weights @ V
```

A two-source calculation you can check

Suppose the unmasked scores are $\ln(3)$ and 0. Softmax gives weights **0.75** and **0.25**. If the values are $[2, 0]$ and $[0, 4]$, the mixed value is **$[1.5, 1]$** . If source 1 is blocked with $-\infty$, its weight becomes 0 and the other weight becomes 1. This is a toy head, before output projection.

Self-attention, cross-attention and heads

Self-attention: query and sources belong to the same token set, as in the image encoder.

Cross-attention: target queries read another set, such as image memory. Multiple heads learn separate projections; their outputs are joined and projected. Small uses 6 encoder heads ($384/6=64$ dimensions each) and 12 decoder heads ($384/12=32$).

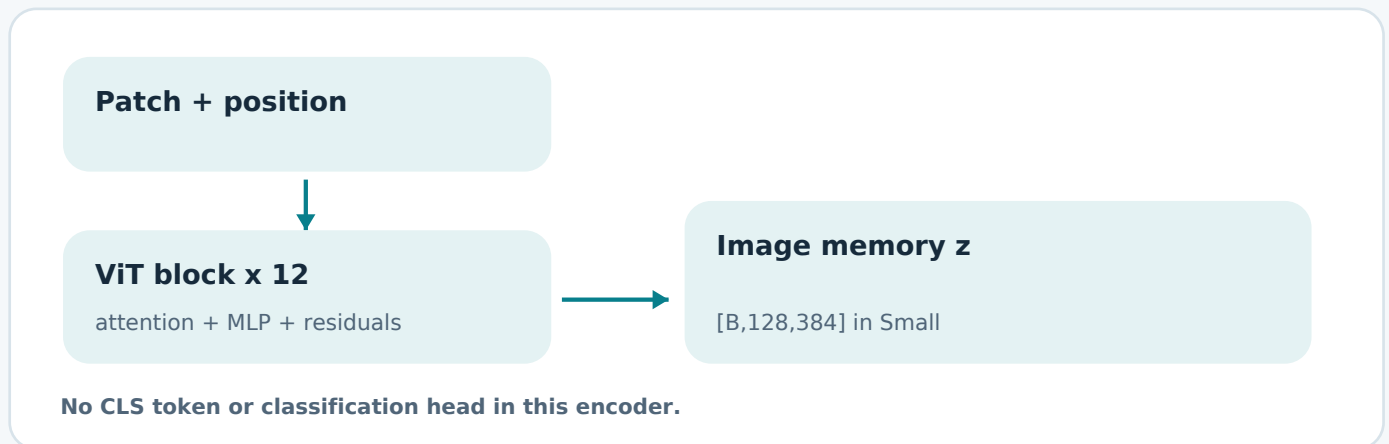
TRY 02 If a source is masked, can its value still contribute directly?

[Answer >](#)

Sources: [P](#) [C3](#)

The encoder builds image memory

One pass creates visual features that decoding can reuse.



What a Transformer block adds

Self-attention lets a patch gather information from other patches. The **MLP** is a small feed-forward network applied to each token: linear expansion, nonlinear activation, linear contraction. A **residual connection** adds the block's input back to its update, so information has a direct path through the stack.

Normalization and training mode matter

LayerNorm (LN) normalizes a token across its feature dimensions using learned scale and bias. **Pre-LN** means normalization happens before the attention or MLP branch. **Dropout** randomly removes some activations during training; evaluation mode disables that randomness. Inference also disables gradient tracking to avoid storing training intermediates.

Global features are not a transcript

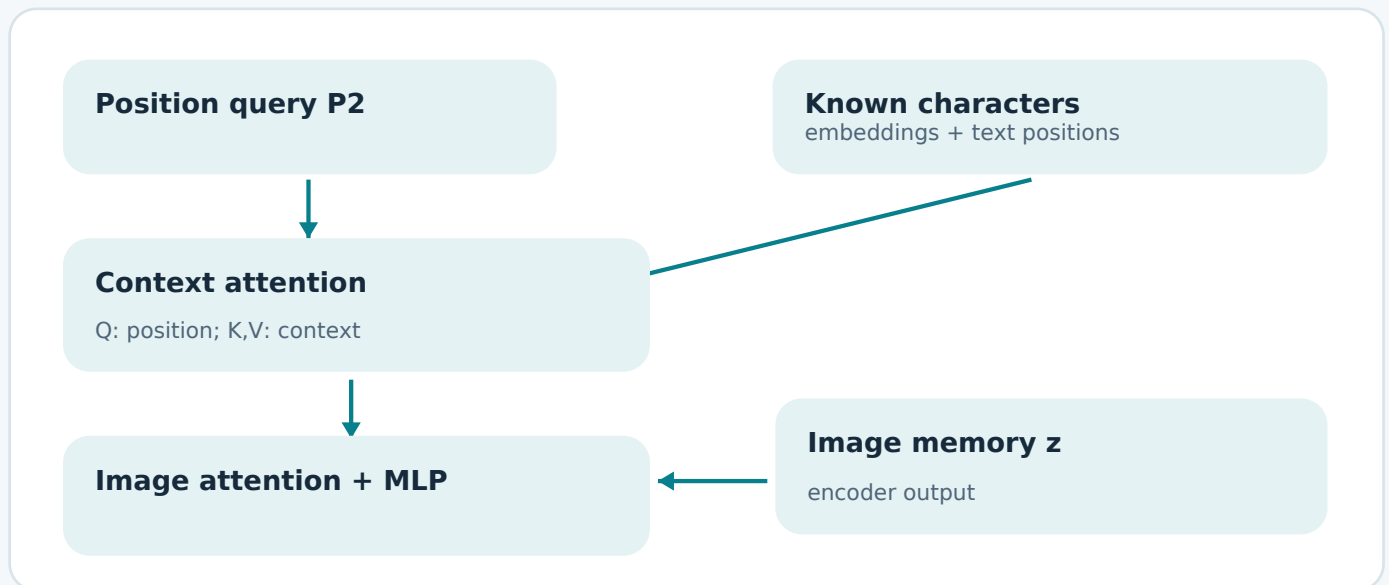
Image memory still consists of visual vectors. It does not already contain a clean word string. The decoder must relate output positions to these vectors. The ViT can distribute a letter's evidence across tokens, so do not interpret each output token as a fixed character box.

A first memory calculation

One Small memory tensor at B=1 has $128 \times 384 = 49,152$ values: **192 KiB in FP32** or **96 KiB in FP16**. This is only one tensor. Peak RAM also includes weights, intermediate activations, runtime workspaces and image buffers.

Inside the decoder

Output position + allowed context + image evidence -> character scores.



Why the position query is separate

P2 identifies the **second output slot** even if only characters at slots 1 and 4 are available. It does not contain the answer 'A'. Context embeddings encode both **which characters are known** and **where those characters belong**. BOS serves as null context and has no added character position embedding.

Three updates, then a classifier

Ignoring LN/dropout for this sketch: $\mathbf{h}_c = \mathbf{p} + \text{MHA}(\mathbf{p}, \mathbf{c}, \mathbf{c}, \text{mask})$; then $\mathbf{h}_i = \mathbf{h}_c + \text{MHA}(\mathbf{h}_c, \mathbf{z}, \mathbf{z})$; then $\mathbf{h} = \mathbf{h}_i + \text{MLP}(\mathbf{h}_i)$. A final linear head maps each D-vector to S+1 logits. Logits are raw scores; softmax turns them into a distribution over character classes including EOS.

One detail that makes code reading easier

The default decoder has **one layer**. Its module is named **self_attn**, but the query stream reads the content stream: the naming does not mean output slots perform ordinary character self-attention. The implementation can update content between deeper decoder layers; the last layer skips that unused update.

Position is not a fixed pixel coordinate

P2 asks for character slot 2, not column 2 of the image. Image attention may read anywhere in the visual memory. Image evidence remains available even when text context is restricted.

Masks encode dependency order

Keep CAT in its real positions. Change who is allowed to see whom.

CONCEPTUAL ALLOW MATRIX: 1 = visible

Order 1 -> 2 -> 3

	B	C	A	T
C / P1	1	0	0	0
A / P2	1	1	0	0
T / P3	1	1	1	0
EOS	1	1	1	1

Order 2 -> 1 -> 3

	B	C	A	T
C / P1	1	0	1	0
A / P2	1	0	0	0
T / P3	1	1	1	0
EOS	1	1	1	1

Illustration excludes the special reverse-EOS training rule.

Read the second matrix as a story

First A is predicted from the image and BOS. Then C can use A. Then T can use C and A. The visible character string has not become ACT: the target rows remain C, A, T. A permutation changes the **conditional factorization**, which means the order in which dependencies are allowed.

Build the rule, not just a triangle

Give each character position a rank in the sampled order. For a target i and context character j , allow j only if **rank(j) < rank(i)**. Always allow BOS. Block the target's own character. This produces the query mask. Padding still needs a separate mask because shorter examples share the batch's sequence width.

The dangerous convention reversal

The visual tables use **1 = allowed**. In the official PyTorch Boolean masks, **True = blocked**. With additive attention scores, **0 = allowed** and **-infinity = blocked**. Never copy an illustration directly into a Boolean mask without checking the receiving API.

Prevent indirect answer leakage

In a deeper decoder, content updates must also follow dependency restrictions. Otherwise a visible token could carry information from a hidden target. The official code creates both content and query masks. Test masks using tiny synthetic sequences before judging training accuracy.

TRY 03 For order 3 -> 1 -> 2, what can C see?

[Answer >](#)

Sources: [C2](#) [C3](#)

Training learns many routes

Teacher forcing supplies correct context; masks control access.

PAPER ABLATION: NAR word accuracy versus training orders K

K=1		0.01%
K=2		22.69%
K=6		92.22%

Table 3: 94-char benchmark, no refinement. Not phone measurements.

What is actually optimized?

For each sampled ordering, predict every target using the permitted **ground-truth** characters as context. This is teacher forcing. Cross entropy for a correct character is **$-\ln p(\text{correct})$** : $p=0.9$ gives about 0.105; $p=0.1$ gives about 2.303. The latter prediction is punished more strongly.

Gradient and optimizer are different jobs

Backpropagation computes gradients: how a small change in each parameter would change the loss. The optimizer uses those gradients to update the parameters. Training repeats this over batches. A gradient is not an explanation of why a particular letter was wrong; it is a local mathematical sensitivity.

Choice	Reference setup / consequence
Permutation count	perm_num=6; include forward order and mirrored sampled orders. Very short labels may have fewer distinct orders.
Shared computation	The image encoder runs once per batch; decoder work repeats for the sampled masks. K is a training cost, not K deployed models.
Loss details	PAD ignored. The implementation accumulates token-weighted CE and stops supervising EOS after the first two permutations.

Paper sketch versus implementation

The code treats EOS specially: forward training gives it full character context, the special reverse ordering gives it null context. This helps teach both contextual and NAR stopping. Permuting characters is not the whole story; inspect **gen_tgt_perms** and **training_step** before reimplementing them.

TRY 04 Does perm_num=6 make every phone inference run six models?

[Answer >](#)

Sources: [P](#) [C2](#) [C4](#)

AR: one new token at a time

The image is encoded once. Predicted characters become context.

BOS	→	C	query P1
BOS C	→	A	query P2
BOS C A	→	T	query P3
BOS C A T	→	EOS	query P4

CAT stays in reading order. Context uses model predictions.

Follow the actual inference loop

Start with BOS. Select the first position query and classify the first character. Append the argmax token ID to the context. Repeat with the next position query. **Argmax** means choose the class with the highest score. This is greedy decoding; it does not search all possible strings as beam search would.

EOS controls variable length

CAT needs three character predictions and a stop prediction. An early EOS loses a suffix; a missing EOS can create junk up to the maximum length. The reference implementation can stop early in testing when every batch member has produced EOS. A long example can keep the whole batch loop active after shorter examples finish.

Error propagation and default refinement

If C is misread as G, later predictions are conditioned on that G. The language context can help or hurt. The default config has **decode_ar=true, refine_iters=1**: the full default behavior is AR followed by one refinement pass. Set refinement explicitly when isolating AR in an experiment.

Cost model to keep in your head

Approximately **one encoder pass + L decoder steps + R refinement passes**, where L includes the stop step and R is refinement count. This is a conceptual model, not a latency prediction. The official AR path uses one target query at a time; it does not provide a standard LLM-style KV-cache interface.

Sources: [C1](#) [C4](#) [C7](#)

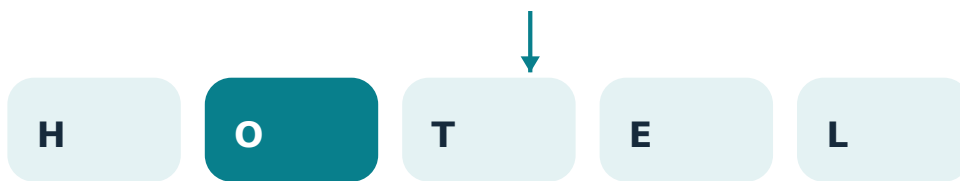
Parallel first, refine together

NAR predicts all slots; cloze refinement reads the other predicted letters.

NAR: all queries read BOS + the same image memory



Refine slot 2 using H _ T E L + image



NAR means no predicted text context on the first pass

With **decode_ar=false**, supply only BOS as context and ask all output positions simultaneously. With a maximum label length of 25 this includes 26 slots. Stop at the first EOS when forming the string. 'No text context' does not mean no global visual information: the encoder has already mixed information across image patches.

Refinement hides the matching token

Take the previous argmax predictions as new context. For target P2, hide the prior character at slot 2 while allowing the other valid characters. All slots update from the **same previous iteration** in parallel. They do not read newly corrected neighbours from the current pass. Image memory stays available and can be reused.

The off-by-one trap

Context columns are **[BOS,y1,y2,...]**; output rows are **[y1,y2,...,EOS]**. To hide y1 for row 0, block context column 1. The cloze 'self' mask is therefore offset in the raw matrix. The implementation also masks EOS and subsequent context slots using the cumulative EOS mask.

Useful experiment, no guaranteed winner

Compare pure NAR, NAR+1/+2 refinement, and AR+1 on the same evaluation crops. Refinement can fix HOTEL, but may also damage a rare SKU. More passes are not automatically better, and a bad initial EOS can restrict useful context.

TRY 05 During one refinement pass, does P3 see the newly fixed P2?

[Answer >](#)

Sources: [C1](#) [C4](#)

Variants: four separate axes

Model size, input geometry, decoding policy and training data all matter.

Configuration	Small / parseq	Tiny / parseq_tiny
Embedding D	384	192
Encoder	12 layers; 6 heads	12 layers; 3 heads
Decoder	1 layer; 12 heads	1 layer; 6 heads
Usual crop / patches	32 x 128 / 4 x 8	32 x 128 / 4 x 8
Visual tokens	128	128

Small and Tiny change width, not the core idea

Tiny uses narrower hidden vectors. Many dense-layer weights scale with D squared: halving D can greatly reduce those weights, but not every component or runtime cost shrinks identically. The default depths stay the same. Tiny is a candidate to benchmark on a phone, not proof that it will meet your feature's accuracy target.

patch16-224 changes image geometry

The official **parseq_patch16_224** experiment uses a 224 x 224 crop and 16 x 16 patches, producing **196 visual tokens**. It inherits the model configuration rather than automatically becoming a larger-width 'Base' model. Square resizing and different patches change the visual representation; weights and preprocessing must match.

Name / flag	What changes
AR / NAR / refine_iters	Inference procedure for the same trained weights. Paper A and N labels are decoding configurations, not separate architecture sizes.
perm_num	Training context diversity. A forward-only checkpoint need not work when switched to NAR.
Real / synthetic training	What image and text distribution the model learned. Same architecture can behave very differently.

Naming discipline

The paper uses PARSeq-S and PARSeq-Ti. An official PARSeq-Base is not implied by ViT naming. Label experiments by checkpoint + input shape + mode + refinement + precision.

TRY 06 Do AR and NAR need two different sets of trained weights?

[Answer >](#)

Sources: [C4](#) [C5](#) [R](#)

Run a trustworthy baseline

Start in Python. Keep a known answer before changing the runtime.

```
import torch
from PIL import Image
from strhub.data.module import SceneTextDataModule

m = torch.hub.load('baudm/parseq', 'parseq',
                  pretrained=True).eval()
tf = SceneTextDataModule.get_transform(m.hparams.img_size)
x = tf(Image.open('crop.png').convert('RGB')).unsqueeze(0)
with torch.inference_mode():
    logits = m(x)
    text, scores = m.tokenizer.decode(logits.softmax(-1))
```

Preprocessing is part of the checkpoint

The reference transform resizes with bicubic interpolation, converts pixels to a tensor, then normalizes using mean 0.5 and standard deviation 0.5. This maps [0,1] pixels to approximately [-1,1]. It resizes to the requested dimensions directly; adding letterbox padding changes that distribution. Verify RGB order, orientation, interpolation, scale and layout in the Android path.

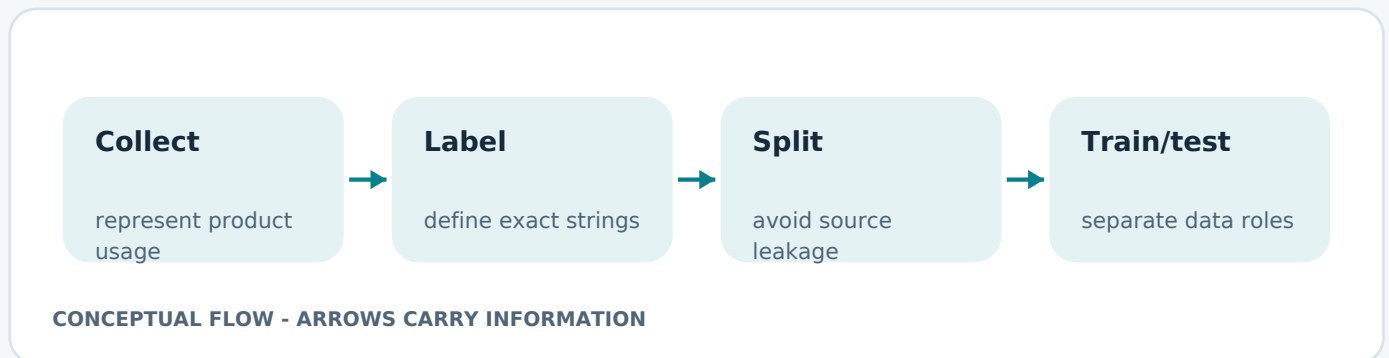
Find this file / symbol	What you learn there
model.py: encode, decode, forward	Actual tensor inputs, queries, AR loop, cloze mask and EOS handling.
system.py: gen_tgt_perms, training_step	Training permutations, PAD/EOS masks and token-weighted loss.
modules.py: DecoderLayer, Decoder	Pre-LN branches, query/content streams and residual updates.
data/module.py + data/utils.py	Image transform, tokenizer IDs, EOS truncation and returned token scores.

Before your first optimization

Keep a small regression pack: clean word, blur, narrow crop, repeated letters, case, punctuation, digits, long word and unreadable crop. Save exact input tensors and FP32 outputs, record commit/checkpoint hashes and resolved config. The snippet is a reference baseline, not a tested phone deployment.

Train for the text you will see

A good network cannot repair a missing symbol or a wrong label policy.



Separate four kinds of data

Training changes model weights. **Validation** selects hyperparameters and checkpoints.

Calibration collects representative activation ranges for quantization. **Test** estimates final quality after choices are fixed. Split by source document, capture session or near-duplicate group so the same text photograph does not leak across sets.

A fine-tuning loop you can defend

Begin with a pinned pretrained baseline. Inspect labels and model crops before training. Use augmentations that resemble your feature: mild blur, noise, lighting, perspective and partial occlusion. Keep labels correct: a crop that removed a character cannot honestly retain the full original word. Tune learning rate and unfreezing on validation results, then evaluate the untouched test set.

Multilingual support is more than a bigger head

Define the charset and Unicode normalization policy first. A visible Hindi glyph may involve multiple code points, so character count, sequence limit and CER need an explicit tokenization policy. New token IDs change embeddings and the classifier. Preserve existing symbol rows deliberately; initialize new rows and train with suitable script data. New labels also change text-context statistics.

Do not silently erase unsupported text

Track labels rejected for unsupported symbols or length limits. Evaluate mixed scripts, numerals, punctuation and rare names separately. If long lines are split into crops, test joins and reading order; the recognizer cannot restore a character cut in half by the upstream crop policy.

First practical check

Overfit a tiny clean set to verify labels, token IDs, masks and loss wiring, then return to the real split. Near-perfect tiny-set results diagnose the training path; they do not establish generalization.

Sources: [R](#) [C2](#) [C7](#)

Accuracy that means something

One overall percentage can hide the failure your feature cares about.

Metric	Definition	What it exposes
Exact word accuracy	Fully correct words / total words	One wrong character fails the word.
Character error rate	(substitutions + deletions + insertions) / reference characters	How much editing is needed; can exceed 100%.
1 - normalized edit distance	Use a declared denominator, e.g. $\max(\text{len}(\text{gt}), \text{len}(\text{pred}))$	Near misses; do not mix conventions.
Acceptance precision / coverage	Correct accepted / accepted; accepted / all	Reliability of a confidence threshold.

A worked example

Reference **HOTEL**, prediction **HOTEL**: word accuracy for this sample is 0%; there is one substitution, so CER is $1/5 = 20\%$. With max-length normalization, 1-NED is **0.8**. Report whether case, punctuation or unsupported characters are normalized away. A phone feature that needs serial numbers may require exact case and punctuation.

Confidence needs validation

The tokenizer returns per-token scores including EOS when present. Their product is length-sensitive and their mean can hide one weak token. Softmax scores are not automatically calibrated probabilities of correctness. Pick a sequence scoring rule, plot accuracy versus score on held-out data, and tune acceptance against a false-accept target.

Slice the benchmark

Break results down by blur, lighting, orientation, word length, script, digits, rare strings, crop quality and device preprocessing path. Compare outputs on paired samples and inspect newly introduced errors. Include confidence intervals or bootstrap uncertainty when a small improvement may be noise.

Fallback is a product policy

For a low-quality crop, retry a better frame or rectification before adding more decoder passes. For low confidence, show uncertainty, request a recapture or use an approved secondary path. Keep the original crop associated with the result while debugging; do not apply an unconditional dictionary correction.

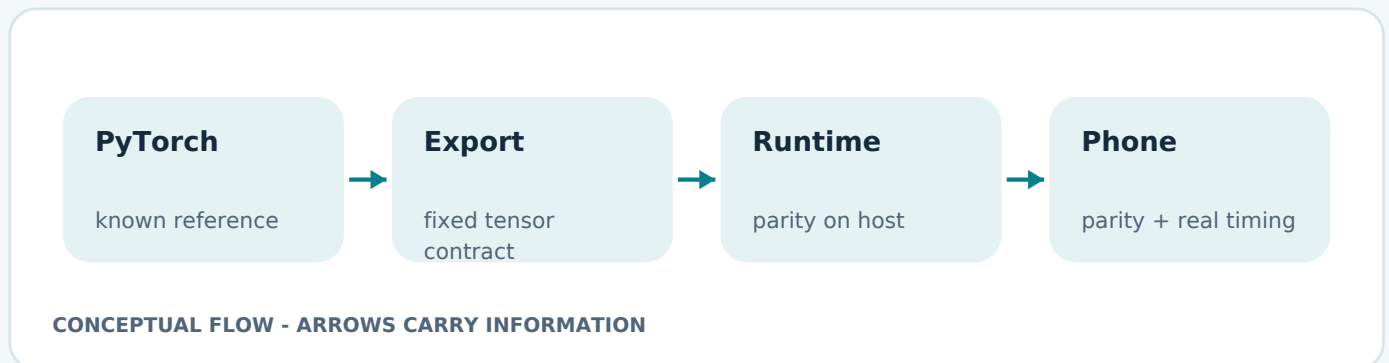
TRY 07 How many word-accuracy points does HOTEL -> HOTEL earn?

[Answer >](#)

Sources: [C7](#) [R](#)

Export with a clear contract

A Python loop and a deployable static graph need explicit contracts.



Begin with the simplest export experiment

A fixed-shape **NAR, refine_iters=0** wrapper is a useful first candidate because it removes the data-dependent AR loop. Then add a fixed number of refinement passes. This is an engineering starting point, not a claim that every PARSeq export succeeds. Verify the converter's operator support and actual output shape before integrating.

A graph is operations plus input/output contracts

An exported graph contains operations such as matrix multiplication, reshape, transpose, normalization and softmax. The runtime must implement each operation with compatible dtype, shape and semantics. LiteRT's current PyTorch path uses LiteRT Torch and requires a torch.export-compatible source model. ONNX Runtime expects an ONNX model. Export success alone says nothing about NPU coverage.

AR has a deployment choice

You can put the loop in the graph if the runtime supports its control flow, or orchestrate decoder steps outside the graph. A proposed split is **encoder -> reusable memory -> decoder step**. Host orchestration may increase per-step dispatch and copy costs. Static unrolling adds fixed work and may weaken early stopping. Measure these choices on the target stack.

Parity ladder

Compare **input tensors first**, then encoder memory, decoder logits, token IDs, EOS position and strings. Use absolute/relative tolerances appropriate to precision, plus exact sequence checks. Small logit drift can change an argmax when top classes are close; a single random input is an inadequate test.

Metadata to ship beside the model

Version the charset/ID order, normalization, input layout/shape, maximum length, mode, refinement count, dtype, checkpoint hash and runtime/provider version together. A mismatched tokenizer can make numerically correct inference produce wrong letters.

TRY 08 If the .onnx file loads, has Android NPU correctness been proven?

[Answer >](#)

Sources: [C1](#) [M1](#) [M3](#)

CPU, GPU, NPU: verify placement

A backend is useful when the whole execution path is useful.

Target	Useful properties	What to verify
CPU	Broad operation coverage; easy reference path	Threads, vector kernels, contention and battery cost.
GPU	Parallel dense computation; FP16 is a candidate	Dispatch, synchronization, copies and shared UI load.
NPU	Potential efficient acceleration on supported silicon	Supported shapes/ops/precision, partitioning and driver behavior.

Delegate / execution provider

A runtime backend that assigns supported graph regions to an accelerator is called a **delegate** in some stacks or an **execution provider** in ONNX Runtime. Unsupported operations may run elsewhere. Inspect partition reports and traces: alternating NPU and CPU regions can add synchronization and conversions that outweigh the arithmetic saved.

Choose the phone and the runtime together

LiteRT provides CPU/GPU/NPU paths with platform and silicon constraints. ONNX Runtime provides CPU and accelerator providers; its QNN provider targets supported Qualcomm stacks. Snapdragon support does not imply equivalent support on Exynos. Identify the actual SoC, OS, driver, runtime version and allowed vendor SDK in your team's build before choosing a route.

Current Android detail

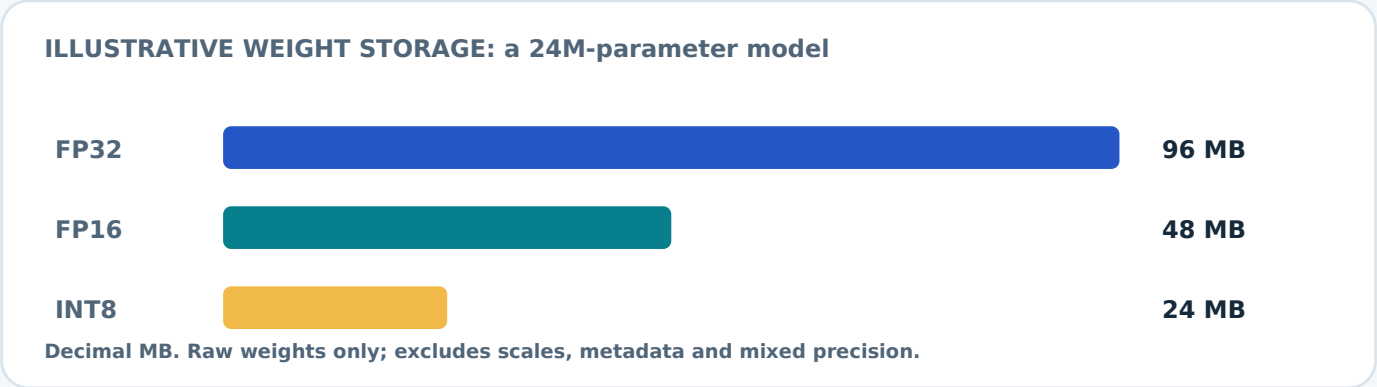
NNAPI was deprecated in Android 15. Do not build a new acceleration plan on the assumption that NNAPI always gives access to the phone's NPU. Some legacy stacks still use it; current options must be checked against the actual device and SDK support. See M5 for the official migration guidance.

Phone integration that survives real usage

Load and warm the model off the UI thread. Reuse inference sessions and buffers where the API permits. Limit queued camera work and discard stale frames. Keep rotation and colour conversion deterministic. Test cancellation, background/resume, memory pressure and concurrent camera/UI work as well as isolated inference.

Precision is a measured trade-off

Smaller numerical formats change storage, arithmetic and sometimes answers.



Understand what INT8 does

Linear quantization approximates a floating value as $\hat{x} = \text{scale} \times (q - \text{zero_point})$. With $\text{scale}=0.02$ and $\text{zero_point}=0$, $x=0.37$ rounds to $q=19$ and reconstructs as 0.38 : the error is 0.01 . Values outside the chosen range are clipped. Too wide a range wastes precision; too narrow a range loses outliers.

Method	What happens	Your check
FP16	Use 16-bit floating weights / computations where supported	Numerical stability and whether kernels accelerate it.
PTQ	Post-training quantization; static activation ranges use calibration data	Use real crops and text contexts representative of the chosen decoding mode.
QAT	Train with simulated quantization effects before conversion	Use when PTQ misses quality targets; validate the final backend kernels.

Weight-only is not full integer execution

Quantized weights may coexist with floating activations. Dynamic quantization chooses activation ranges during inference; static quantization fixes them using calibration. Backend support determines efficiency. File shrinkage and speedup are separate measurements.

Localize quality loss

Compare FP32 and quantized tensors on the same crops: layer drift, EOS changes and rare/digit errors. Test mixed precision for sensitive normalization, attention or logits where supported. Accept using held-out sequence quality, not only tensor similarity.

TRY 09 Does 4x smaller weight storage guarantee 4x faster OCR? [Answer >](#)

Sources: [M2](#) [M4](#)

Profile the whole phone feature

The slowest stage may be outside the recognizer.

HYPOTHETICAL SERIAL TRACE - not device measurements



Amdahl intuition: optimize what dominates

If decoder time in this toy trace is halved from 6 to 3 ms, total time drops from 43 to **40 ms**, only a **1.075x** speedup. Model-only improvements can look impressive while the feature hardly changes. Instrument capture/queueing, detector, crop conversion, encoder, decoder, transfers and postprocessing separately.

Latency, throughput, RAM and energy are different

Report warm p50 and p95 latency, cold load/compile time, crops per second, model storage, peak memory and energy per useful result. In a purely serial 43 ms loop, the reciprocal is about 23.3 results/s; an overlapping pipeline can behave differently. For sustained runs, control brightness, input stream, warm-up, charging state and ambient conditions; observe thermal throttling.

Optimization order you can test

First fix unnecessary copies and session reloads. Reuse image memory across AR/refinement. Compare decoding modes and Tiny. Then test FP16/INT8 and graph fusion. **Distillation** trains a smaller student using a teacher's targets or features. **Structured pruning** removes useful hardware-sized groups such as channels or layers; scattered zeros may not accelerate dense kernels.

Make the experiment attributable

Change one factor, keep paired crops and record config/device/runtime. Synchronize asynchronous backend completion before stopping timers. Separate model microbenchmarks from end-to-end traces. Test 1, 10 and 40 text regions plus short/long labels; crop count can multiply recognizer cost.

TRY 10 In the toy trace, what happens if decode time becomes 3 ms?

[Answer >](#)

Sources: [C1](#) [M1](#) [M2](#)

Diagnose, then check your answers

Make a falsifiable hypothesis before changing the model.

Observed failure	First test	Likely next experiment
Python correct; Android wrong	Compare the exact normalized input tensor	Check RGB, layout, orientation, scaling and resize parity.
Only INT8 loses digits / suffixes	Compare first divergent logits and EOS positions	Broaden calibration contexts; try targeted mixed precision or QAT.
Cold run fast; sustained run slow	Trace frequencies, thermal state, queue depth	Test lighter mode/model and lower work rate; confirm backend placement.

CHECKPOINT ANSWERS - tap a number to return

- 01** One batch item, up to 26 output slots, 95 class scores per slot.
- 02** No. A blocked source has zero attention weight in that operation.
- 03** BOS and T. Order 3 -> 1 -> 2 puts T before C, but A after it.
- 04** No. Six is sampled training orders; deployment uses one shared model.
- 05** No. Every slot reads the previous iteration, so updates are synchronous.
- 06** No. A PLM-trained checkpoint supports both; test each mode explicitly.
- 07** Zero for that word. One substitution still makes the whole word wrong.
- 08** No. Prove conversion parity, device parity and actual backend placement.
- 09** No. Compute, memory traffic, kernel support and transfers determine speed.
- 10** 40 ms total; $43/40 = 1.075\times$ speedup, about 7% lower latency.

Three misunderstandings to retire

The decoder does not erase visual ambiguity by magic. NAR does not bypass the encoder. A supported accelerator name does not prove that your graph ran there. Each claim should correspond to a tensor comparison, evaluation slice or execution trace.

Your first engineering lab

A concrete learning loop: baseline -> compare -> export -> measure.

- ☐ **Baseline:** Run 20 diverse crops; record exact inputs, outputs and versions.
- ☐ **Modes:** Compare NAR, NAR+1/+2 and AR+1 with the same checkpoint.
- ☐ **Errors:** Inspect EOS, crop failures, digits and rare strings separately.
- ☐ **Export:** Prove fixed-shape host parity, then device parity.
- ☐ **Device:** Measure cold, warm and sustained behavior plus memory.

Done means: a teammate can reproduce your config, input path, quality result and timing conditions. Agree feature-specific accuracy/latency/memory budgets with the team; do not invent a universal flagship target.

PRIMARY SOURCES - every row is clickable

- P** Original paper: method, experiments, appendices
- R** Official repository: inference and evaluation
- C1** Model: encode, decode, AR and refinement
- C2** Training: permutations, masks, EOS and loss
- C3** Modules: query/content streams and pre-LN
- C4** Default Small configuration
- C5** Tiny and patch16-224 configurations
- C6** Image transforms
- C7** Tokenizer and EOS truncation
- M1** ONNX Runtime mobile deployment
- M2** ONNX Runtime quantization and debugging
- M3** LiteRT PyTorch conversion
- M4** LiteRT: supported acceleration paths
- M5** Android: NNAPI deprecation
- M6** ONNX Runtime QNN provider
- T1** CTC: sequence alignment and blank collapse
- T2** ABINet: bidirectional language refinement
- T3** TrOCR: pretrained image/text Transformers

Reference configuration/code checked against official main-branch files on 7 October 2026. Pin a commit for your own experiments. Diagrams are original teaching illustrations; toy timings and storage examples are explicitly hypothetical. This guide does not assert Samsung implementation details or measured phone results.